

Speeding up network motif discovery using subtree counting*

Joshua A. Grochow[†] and Tarek Tohme[‡]

Abstract. We propose a new algorithm for counting the number of subgraphs of a network isomorphic to a given query graph, in the context of network motif search. Vertices of large degree (sometimes called hubs) contribute to a combinatorial explosion in the total number of subgraphs of a network. Interactome networks and scale-free networks tend to possess such high-degree vertices, which translates in motif search being intractable for graph sizes greater than 8 on a wide variety of networks of interest. Our procedure leverages the k -core decomposition and a novel subtree-counting technique to quickly scan the periphery of a network, producing significant speedup in the presence of trees and peripheral hubs. Namely, we provide a reduction from counting isomorphic rooted subtrees to computing the permanent of a weighed biadjacency matrix. We provide analytic upper bounds on the algorithm’s execution time and test it on a corpus of real network data.

Key words. network motifs, subgraph isomorphism, k -core decomposition

AMS subject classifications. 05C85, 05C30, 05C60

1. Introduction. A network is a useful abstraction for studying structural patterns and properties in many types of systems. One way to study such patterns is by identifying repetitions of certain subnetworks, following the reasoning that in many systems, the frequent occurrence of a “module” is an indicator of its importance in the function of the system. Patterns in a network that appear with statistically significant frequency with respect to a null model (an appropriately defined random network) are called *network motifs*. Examples of networks where motifs have been studied include class diagrams in software architecture , brain networks , transcriptional regulation networks, and many more .

Since their introduction , an active line of research has been dedicated to the design of algorithms that search for such motifs ever more efficiently. Most of these algorithms fall under one of two categories: network-centric algorithms, in which all or a sample of size- k subgraphs of a given network are enumerated, and motif-centric algorithms in which size- k graphs are enumerated *a priori* and mapped onto a network one by one. Typically in the former category, after enumeration, subgraphs are grouped into isomorphic equivalence classes and their frequencies are then calculated or estimated. Two of the advantages of motif-centric approaches is the possibility of searching for the instances of a single subgraph in a network, and the reduced cost of computing isomorphisms for each subgraph incurred in the grouping task of network-centric algorithms. Additional ideas such as symmetry-breaking and expansion trees have enabled further speedups, making it possible to search for all subgraphs up to size 8, and individual subgraphs up to size 15 . The g-trie, introduced in is an efficient data structure that enables the search for any selection of subgraphs, taking advantage of the common subgraphs within that selection. This minimizes the cost of redundant searches

*Submitted to the editors on DATE

Funding: Funding information

[†]Departments of Computer Science and Mathematics, University of Colorado, Boulder (jgrochow@colorado.edu).

[‡]Department of Computer Science, University of Colorado, Boulder (tarektohme1@gmail.com).

incurring by motif-centric approaches, which execute as many queries as there are subgraphs disregarding their potential similarity, while eliminating the unnecessary isomorphism checks of network subgraph enumeration. This method used in combination with symmetry-breaking produces a vast improvement compared with pre-existing approaches. . Additional approaches have included focusing only on tree motifs parallelizing the search using GPUs .

A few remarks serve to lay out our strategy for an improved motif search algorithm. First, we note that existing approaches to motif search all operate by either enumerating or mapping *instances* of subgraphs inside a network. In principle, to discover motifs it would suffice to count all subgraphs of a network, rather than list them. The counting version of the subgraph isomorphism problem, $\#SUBGRAPHISOMORPHISM$, asks how many subgraphs of a graph G are isomorphic to another graph H . Algorithms for $\#SUBGRAPHISOMORPHISM$ include and . The latter color-coding algorithm achieves polynomial time when counting non-induced subgraphs of bounded treewidth, provided $|H| = O(\log(|G|))$. However it was recently proved that boundedness of the vertex cover number of H is the only condition for tractability of $\#SUBGRAPHISOMORPHISM$. Regardless of asymptotic behavior, it is clear that counting subgraphs is faster than listing them, simply owing to the memory access and storage overhead when large numbers of graphs are involved.

Another issue with existing algorithms is that none of them account for the difference in runtime incurred by different subgraph topologies. Early on in the investigation of network motifs, it was found that certain subgraphs contribute disproportionately more to the runtime of a motif search algorithm because of combinatorial explosions due to their shape. A good example is the star graph of size $k + 1$, of which there are $\binom{n}{k} \geq (\frac{n}{k})^k$ instances in a star graph of size $n + 1$.

Lastly, scale-free networks are known to possess large hierarchically structured hubs which can quickly make the motif search problem intractable in the contexts where they show up. Putting aside the discussion of the ubiquity of such networks in the real world, many biological networks, where investigations of network motifs originated, often also contain prohibitively large hubs. As we will check experimentally, the runtime of motif search can sometimes be dominated by a single star motif while searching among hundreds or thousands of motifs, especially when the target network contains large hubs. An obvious remedy to this is to simply compute $\binom{n}{k}$ whenever we encounter such a situation.

Taking into account the above remarks, we make $\#SUBGRAPHISOMORPHISM$ our focus in the present paper, and take advantage of both motif and network topology to accelerate subgraph counting. We show that a special case of this problem where the network G and query H are rooted unlabeled trees is $\#P$ -complete. We then build on Grochow-Kellis to design a subgraph-counting algorithm that leverages tree and hub structure in a network to efficiently count all instances of a subgraph. Our main contribution consists in separating each of the network and the query into two subsets using the k -core decomposition: the “periphery”, which contains all the low-connectedness nodes, and the more interconnected nodes, which can be viewed as “the core”. A key feature of this decomposition is that it guarantees that the periphery consists entirely of trees. This enables us to run the peripheries of both network and query through a fast subtree-counting algorithm, saving considerable time compared with standard motif search procedures. In addition, the same decomposition offers a powerful early abort for motif search; the *coreness* of a vertex imposes a strong constraint on the possible

isomorphisms that can map a query graph to a target network, allowing our algorithm to waste less time trying out mappings that do not yield valid instances.

In section 2 we make the notions of core and periphery rigorous, analyze the time complexity of subtree-counting and provide an algorithm to that end. Section 3 introduces our main algorithm and provides an analytic treatment of its performance. We then test the algorithm on a corpus of real networks, and relate its empirical performance to several key network statistics in section 4. We conclude in section 5.

2. Counting subtrees in the 1-shell.

2.1. The k -core decomposition. The k -core decomposition is a procedure that reveals the “core-periphery” structure of a given network. A k -core is defined as the maximal subnetwork that contains only vertices of degree at least k . The k -shell is the set of vertices in the k -core that are not part of the $k+1$ -core. A vertex inside the k -shell is said to be of *coreness* k . The *degeneracy* of a network is the maximal coreness of any of its vertices. The k -core decomposition can be carried out in linear time as follows: remove all vertices of degree 1 until there are none left. Those vertices constitute the 1-shell. Make additional passes removing all vertices of degree at most k at each pass, labeling each vertex with its coreness k .

In any network with degeneracy 2 or more, the 1-shell consists entirely of trees, each of whose root belongs to the 2-core. To see why, notice that no tree can contain a nontrivial subgraph where every vertex is of degree 2 or more. Performing a k -core decomposition on H and G reveals these trees, if they exist. By selecting one root at a time from each of H and G 's 1-shell trees, we can then run a subtree-counting algorithm as a subroutine to any standard motif-centric algorithm. We build on the Grochow-Kellis symmetry-breaking algorithm, but in principle our subtree-counting technique can be used with any standard motif-centric algorithm, with minor modifications. In the following we discuss the computational complexity of subtree-counting and motivate its use in the context of motif search.

2.2. Complexity of counting subtrees. Given two rooted, unlabeled trees T and S , we define $\# \text{ROOTEDSUBTREES}$ as the problem of counting the number of subtrees of S isomorphic to T . This problem is $\# \mathbf{P}$ -complete, as we prove in Theorem 2.1 via a reduction from $\# \text{IMPERFECTMATCHINGS}$, the problem of counting the number of matchings in a bipartite graph. The latter problem was proved $\# \mathbf{P}$ -complete by Valiant. As in Valiant, we define a counting Turing machine as a standard nondeterministic TM that can output the count of its accepting configurations on an auxiliary tape. We write $A \leq_T B$ if language A is Turing-reducible to language B , $T \preceq S$ if there exists a root-preserving isomorphism between tree T and a subtree of S , and $T \preceq! S$ if such an isomorphism is unique up to the symmetries of T .

Theorem 2.1. $\# \text{ROOTEDSUBTREES}$ is $\# \mathbf{P}$ -complete.

Proof. Let S and T be two rooted, unlabeled trees. Membership in $\# \mathbf{P}$ is easy to verify: consider a counting Turing machine that removes leaves from S nondeterministically and checks for tree isomorphism with T in polynomial time using preorder strings. The number of accepting configurations is equal to the number of instances of T in S .

Let $B = (L, R, E)$ be a bipartite graph where L and R are respectively the sets of left and right vertices, and $E = \{(u, v) | u \in L, v \in R\}$ is the set of edges. To show that $\# \text{IMPERFECTMATCHINGS} \leq_T \# \text{ROOTEDSUBTREES}$, we construct a tree T_u for each vertex

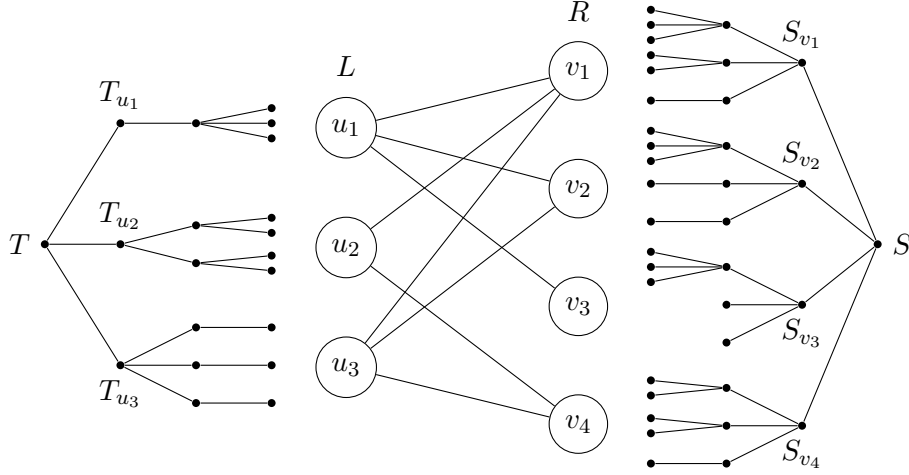


Figure 1. Graphical example of equivalence (2.1).

124 $u \in L$, and a tree S_v for each $v \in R$ (see Figure 1), such that

$$125 \quad (2.1) \quad (u, v) \in E \iff T_u \preceq! S_v$$

126 On the left side of the bipartite graph, we then create a tree by adding an edge between
 127 a new vertex t and the root of T_u , for each $u \in L$. Every T_u is now a subtree of the new
 128 tree T , rooted at a child vertex of t . Similarly, we create tree S on the right side. This
 129 establishes a one-to-one correspondence between a matching of B and an instance of T in S
 130 (up to symmetry).

131 We construct the $\{T_u\}_{u \in L}$ trees as follows: $\forall i \in \{1, \dots, |L|\}$, T_{u_i} has i depth-1 vertices,
 132 and each of those has $|L| - i + 1$ children. For the $\{S_v\}$ trees, $\forall j \in \{1, \dots, |R|\}$, S_{v_j} has exactly
 133 $|L|$ depth-1 vertices. We then populate the children of those vertices by “inserting” one copy
 134 of T_{u_i} into S_{v_j} if and only if $(u_i, v_j) \in E$; in other words, i among S_{v_j} ’s depth-1 vertices has
 135 at least $|L| - i + 1$ children, and the remaining $|L| - i$ depth-1 vertices have no children. This
 136 construction guarantees that at most one instance of T_{v_i} appears in S_{v_j} .

137 Now if a Turing machine can query an oracle for $\# \text{ROOTEDSUBTREES}$ on instance (T, S) ,
 138 the answer is equal to the number of bipartite matchings in graph B , thus establishing the
 139 result. ■

140 **2.3. Description of the subtree-counting algorithm.** Valiant showed that the number
 141 of matchings of a bipartite graph is equal to the permanent of its adjacency matrix. We
 142 use that fact and the above reduction to propose a subtree-counting algorithm that leverages
 143 Ryser’s formula for the permanent to achieve drastic speedup when compared with regular
 144 motif search algorithms running on trees.

145 **Algorithm 2.1** counts all the instances of an unlabeled tree T inside an unlabeled tree S ,
 146 where the roots of both trees are given by t and s respectively. D is a dictionary that stores
 147 at key u the preorder traversal string of the subtree of T rooted at u for all nodes $u \in T$ (here
 148 we abuse the term “subtree” by having it refer to the subgraph of T that contains u and all
 149 its descendants relative to the root of T).

Algorithm 2.1 COUNTROOTEDSUBTREES(T, S, t, s, D)

```

subtreesPreorderStrings :=  $\{D[i] \mid \forall i \in t.children\}$ 
equivalenceClassSizes :=  $\{\text{countStringsEqualTo}[k] \mid \forall k \in \text{subtreesPreorderStrings}\}$ 
multiplicity := 1
for  $n$  in equivalenceClassSizes do
    multiplicity := multiplicity  $\times n!$ 
end for
if  $t.children = \emptyset$  then
    return 1
end if
for  $u$  in  $t.children$  do
    for  $v$  in  $s.children$  do
         $T_u := \text{detachTree}(T, u, t)$  //the “downward-facing” subtree of  $T$  rooted at  $u$ 
         $S_v := \text{detachTree}(S, v, s)$ 
         $k := \text{COUNTROOTEDSUBTREES}(T_u, S_v, u, v, D)$ 
         $M[u][v] := k$ 
    end for
end for
return  $\text{permanent}(M) \div \text{multiplicity}$ 

```

150 We are interested in counting only one copy of T inside S for each one of T 's automorphism
151 classes, since we are dealing with unlabeled trees where symmetries do not matter. To account
152 for this, the algorithm starts by listing the preorder strings of the depth-1 subtrees of T , and
153 grouping the identical strings among that list into equivalence classes (recall that a preorder
154 traversal uniquely determines an unlabeled tree, up to symmetry). There are $n!$ possible
155 permutations of the elements of an equivalence class of subtrees, whence the correction factor
156 *multiplicity*.

157 Next, the algorithm checks for the base case, where the tree T consists of only one node
158 with no children. Otherwise, it loops through every pair of subtrees (T_u, S_v) rooted at depth-1
159 and recursively counts the instances of T_u in S_v , storing the count in a matrix M at entry
160 $[u][v]$. This is equivalent to constructing a bipartite graph and assigning a weight k to edge
161 (u, v) . Finally, the permanent of M is computed using Ryser's formula:

$$162 \quad (2.2) \quad \text{perm}(M) = \sum_{k=0}^{n-1} (-1)^k \Sigma_k$$

163 where Σ_k is obtained by summing the products of the row-sums of A_k , the matrix obtained
164 by deleting k rows from M , for all combinations of rows. The worst-case time complexity
165 of using Ryser's formula for the permanent is $O(2^{n-1}n^2)$. However, to get an idea of how
166 much faster [Algorithm 2.1](#) is than regular motif search over trees, see **(add empirical speed**
167 **comparison)**.

168 3. The subgraph-counting algorithm.

3.1. Boundary conditions. Recall from the introduction that our method consists in separating the 1-shell from the 2-core and performing subtree-counting on the former, then running a regular motif search algorithm on the latter. In doing so we must take care of a few boundary cases that arise at the interface between the 1-shell and the 2-core (**add figure to visualize 1-shell and 2-core**). To see why, let $H = (C_H, P_H)$ be a query graph and $G = (C_G, P_G)$ be a network, where C and P refer respectively to the 2-core and the 1-shell. An instance of H could appear “halfway” between the 1-shell and the 2-core of G such that only part of P_H extends into P_G . Because of this, to properly count every instance of H in G , we must run each tree $T_H \in P_H$ as well as all of its subtrees through [Algorithm 2.1](#). By using the symmetry-breaking technique introduced in on T_H , we can cut down the cost incurred by enumerating all possible subtrees by an amount proportional to the product of the number of symmetries at each level of trees of P_H .

On the flip side, another boundary condition provides us with a powerful early abort when searching for instances of H inside G : suppose that H appears halfway between C_G and P_G , only this time C_H extends partially into P_G . As we prove below, this cannot happen. In fact, it cannot happen between a k -core in H and an h -shell in G for any integers $h < k$ without violating induced isomorphism, which we require in the formal definition of “an instance of H inside G ”.

Definition 3.1. We say that an instance of $H = (V_H, E_H)$ appears in $G = (V_G, E_G)$ when there exists an induced isomorphism $\phi : V_H \mapsto V_G$ between the nodes of H and those of G :

$$(3.1) \quad \forall u, v \in V_H, (u, v) \in E_H \iff (\phi(u), \phi(v)) \in E_G$$

Theorem 3.2. For any k , a mapping between a vertex $v \in H$ of coreness k onto a vertex $u \in G$ of coreness $h < k$ violates induced isomorphism between H and G .

Proof. First notice that a vertex v in a graph’s k -shell can have at most k neighbors in the $k + 1$ -core. Otherwise, it wouldn’t be removed when the k -core decomposition is making passes to remove vertices of degree at most k , and hence would not be part of the k -shell by definition. By the same reasoning, a vertex u in the $k + 1$ -core must have at least $k + 1$ neighbors inside the $k + 1$ -core. The above definition implies that an isomorphism that maps u to v should also map at least one of u ’s $k + 1$ -core neighbors to a neighbor of v in the k -shell. Proceeding by induction, we see that v has at least $k + 1$ neighbors, each of which has at least $k + 1$ neighbors, and so on. Therefore v must be in the $k + 1$ -core, a contradiction. ■

We can use this fact to create an early abort by coreness: our algorithm should never bother to try to map a vertex in H to another in G with lesser coreness.

3.2. Main result. We build on the symmetry-breaking algorithm introduced by Grochow and Kellis, and refer to the algorithms from their paper by the same name. COUNTSUBGRAPHS ([Algorithm 3.1](#)) takes in two graphs H and G , and returns the number of images of H that appear in G . It starts by running a k -core decomposition on both graphs. Then, as in the Grochow-Kellis algorithm, it finds all automorphisms from H onto itself by calling FINDSUBGRAPHINSTANCES(H, H), groups the nodes that map to each other in the same equivalence class and returns a representative node from each class. We abbreviate this behavior in the function *findEquivalenceClassRepresentatives*. The symmetry-breaking conditions

are also computed similarly as in Grochow-Kellis.

The next part of the algorithm consists in listing all the subtrees of the 1-shell of H , up to symmetry. This is done by the function *enumerateSubtrees*, which stores them in the dictionary *subtrees*, keyed by the nodes in H at which they are rooted. It does this by again making use of symmetry-breaking, and listing only one tree from each equivalence class of symmetries.

The algorithm proceeds to count the subtrees of H that appear in G 's 1-shell and at the boundary with its 2-core. It loops through all of G 's nodes in the same order as the k -core decomposition to quickly scan the 1-shell nodes and then remove them, making the subsequent 2-core search more tractable. We make use of the function f , which represents the partial isomorphism between a subset of H and G as the algorithm progresses in its search. For the boundary case, we associate each f with a multiplier, which simply consists of a count of the number of subtrees that "extend" from f into the 1-shell of G . ISOMORPHICEXTENSIONS is a function introduced in Grochow-Kellis which incrementally extends f one vertex at a time until it maps the entirety of H onto a subset of G . By using the multiplier, we can initialize f from a root of G and let it expand inward into the 2-core, and use COUNTROOTEDSUBTREES to count partial instances of H in the 1-shell. At the end of the search, we multiply the number of instances found by ISOMORPHICEXTENSIONS by their individual multipliers to recover the total number of boundary instances. The vertices scanned by the subtree-counting routine are marked to prevent f extending into the 1-shell. After all the 1-shell vertices are taken care of, they are removed and only the 2-core of G remains.

Note that here we use a slightly modified version of ISOMORPHICEXTENSIONS that accounts for the early abort by coreness feature discussed above. Namely, when searching for an image vertex $g \in G$, ISOMORPHICEXTENSIONS first checks the coreness of g to ensure that it is greater or equal than the coreness of its tentative preimage h .

3.3. Upper bounds on runtime. The subtree-counting algorithm works by recursively computing the number of matchings at each level of a query tree T and a target tree S . To obtain an analytic upper bound on the number of steps in its execution, we count the total number of calls to COUNTROOTEDSUBTREES, and multiply that by the worst-case runtime of computing the permanent of a weighed adjacency matrix. Note that in general such a matrix will not be square, which leads to an added binomial coefficient in the permanent's runtime contribution. This is because the permanent of a non-square matrix is the product of the permanents of each of its square submatrices.

The following is an approximate expression of the number of steps in COUNTROOTEDSUBTREES' execution on trees T and S . T' is the set of non-leaf vertices of T , ϕ is the root-preserving isomorphism mapping T to a subtree of S , m_u and n_v are the number of children of vertices $u \in T$ and $v \in S$ respectively:

$$(3.2) \quad f_{crs}(T, S) \approx \sum_{u \in T'} \binom{n_{\phi(u)}}{m_u} \times m_u^2 \times 2^{m_u-1}$$

By comparison, the Grochow-Kellis algorithm will test all m vertices of T with all n vertices of S up to symmetries of T .

Algorithm 3.1 COUNTSUBGRAPHS(H, G)

```

//Initialization
kCoreDecompose(H)
kCoreDecompose(G)
 $ECR_H := \text{findEquivalenceClassesRepresentatives}(H)$ 
 $SBC_H := \text{findSymmetryBreakingConditions}(H)$ 
 $f := \text{initializeParitalMap}$ 

//enumerating all subtrees in  $H$ 's 1-shell
for  $h$  in  $ECR_H$  do
  if  $h \in P_H$  OR  $h$  is a root then
     $T := \text{detachTree}(H, h[, \text{parent}(h)])$ 
     $\text{subtrees}[h] := \text{enumerateSubtrees}(T, h)$ 
  end if
end for

//counting subtrees in  $G$ 's 1-shell and boundary
for  $g$  in kCoreTraversalOrder( $G$ ) do
  if  $g \in P_G$  AND  $H$  is a tree then
     $S := \text{detachTree}(G, g, \text{parent}(g))$ 
    for  $h$  in  $ECR_H$  do
      1ShellCount := COUNTROOTEDSUBTREES( $H, h, S, g$ )
    end for

  else if  $g$  is a root then
     $S := \text{detachTree}(G, g)$ 
    for  $h$  in  $ECR_H$  do
      for  $T_h$  in  $\text{subtrees}[h]$  do
        boundaryMultiplier := COUNTROOTEDSUBTREES( $T_h, h, S, g$ )
         $f.\text{setMultiplier}(\text{boundaryMultiplier})$ 
         $G.\text{markNodes}(S)$ 
        ISOMORPHICEXTENSIONS( $f, H \setminus T_h, G, SBC_H$ )
      end for
    end for
     $G.\text{removeNodes}(S)$ 
  end if
end for

//counting subgraphs in the 2-core
2CoreCount := count(FINDSUBGRAPHINSTANCES( $H, G$ ))
boundaryCount :=  $\prod_{f \in \text{boundary}} f.\text{getMultiplier}$ 

return 1ShellCount + boundaryCount + 2CoreCount

```

$$(3.3) \quad f_{gk}(T, S) \approx \frac{1}{|Aut(T)|} \binom{n}{m} = \frac{1}{|Aut(T)|} \binom{1 + \sum_{u \in T'} n_{\phi(u)}}{1 + \sum_{u \in T'} m_u}$$

In the context of counting subgraphs, COUNTROOTEDSUBTREES will be called once for each pair of non-leaf vertices in H and roots in G . To factor in the speedup afforded by the core-ness condition described in [Theorem 3.2](#), let $m = |H|$ and $n = |G|$, let m_K, n_K be the sizes of the k -cores of H and G , and R the number of roots in G . This gives an upper bound on the total runtime of the subgraph counting algorithm:

$$(3.4) \quad f_{cs}(H, G) \leq \frac{R}{|Aut(H)|} \times \sum_{T \subseteq P_H} f_{crs}(T, S) \times \binom{n_K}{m_K} \binom{n_{K-1} - m_K}{m_{K-1} - m_K} \cdots \binom{n_2 - m_3}{m_2 - m_3}$$

We compare this with an upper bound on the Grochow-Kellis algorithm's performance:

$$(3.5) \quad f_{gk}(H, G) \leq \frac{1}{|Aut(H)|} \binom{n}{m}$$

4. Real-data experiments.

4.1. Methods. We implemented the Grochow-Kellis algorithm and [Algorithm 3.1](#) in Python and compared the runtime incurred in counting all undirected graphs up to size k .

4.2. Results and discussion.

5. Conclusion.

REFERENCES

- [1] T. G. KOLDA AND J. R. MAYO, *An adaptive shifted power method for computing generalized tensor eigenpairs*, SIAM Journal on Matrix Analysis and Applications, 35 (2014), pp. 1563–1581, <https://doi.org/10.1137/140951758>.
- [2] T. G. KOLDA AND J. R. MAYO, *An adaptive shifted power method for computing generalized tensor eigenpairs*, SIAM Journal on Matrix Analysis and Applications, 35 (2014), pp. 1563–1581, <https://doi.org/10.1137/140951758>.